

Laporan Praktikum 2 Kontrol Cerdas

Nama : Ahimsa Raihan Alwikan Abhipraya

NIM : 224308002

Kelas : TKA 6A

Akun Github (Tautan) : <https://github.com/ahimsarai>

Student Lab Assistant : Muhammad Fatih Anfa Adhima

1. Judul Percobaan

Implementasi Machine Learning untuk Deteksi Warna Secara Real-Time Menggunakan Decision Tree dan KNN dengan OpenCV

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

- Memahami penerapan algoritma Decision Tree dan K-Nearest Neighbors (KNN) untuk melakukan klasifikasi warna.
- Melatih model machine learning dengan memanfaatkan dataset warna dalam bentuk CSV dan gambar.
- Melakukan normalisasi data guna meningkatkan akurasi model yang dihasilkan.

3. Landasan Teori

- **Machine Learning untuk Klasifikasi Warna** : Machine Learning (ML) digunakan untuk memetakan nilai warna (RGB) ke kategori warna tertentu. Model ML mempelajari pola data dan mengklasifikasikannya secara otomatis.
- **Algoritma Decision Tree** : Decision Tree bekerja dengan membagi data berdasarkan fitur (nilai RGB) hingga mencapai keputusan akhir. Algoritma ini mudah dipahami dan efektif untuk klasifikasi warna dengan struktur keputusan yang jelas.
- **Algoritma K-Nearest Neighbors (KNN)** : KNN mengklasifikasikan warna berdasarkan K tetangga terdekat menggunakan jarak Euclidean. Algoritma ini sederhana dan efektif, tetapi memerlukan data yang telah dinormalisasi untuk hasil yang akurat.
- **Normalisasi Data** : Normalisasi mengubah skala data RGB (0–255) ke rentang 0–1 untuk mencegah fitur tertentu mendominasi dan meningkatkan akurasi model, terutama pada algoritma berbasis jarak seperti KNN.
- **OpenCV untuk Pemrosesan Citra** : OpenCV digunakan untuk mengakses kamera, menangkap frame real-time, dan memproses gambar. Konversi warna dari BGR ke RGB dilakukan agar sesuai dengan format yang digunakan dalam model ML.
- **Deteksi Warna Real-Time** : Proses deteksi mencakup pengambilan gambar, ekstraksi nilai RGB, normalisasi data, prediksi warna menggunakan Decision Tree atau KNN, dan visualisasi hasil secara langsung di frame kamera.

4. Analisis dan Diskusi

A. Bagaimana Performa Model dalam Mendeteksi Warna?

Performa model dalam deteksi warna diukur berdasarkan **akurasi**, **presisi**, dan **waktu prediksi real-time**.

- **Decision Tree:** Cepat dan mudah dipahami, namun dapat overfitting jika data terbatas.
- **KNN:** Akurat untuk dataset kecil, tetapi prediksi lebih lambat pada dataset besar karena perhitungan jarak.
- **Faktor yang Mempengaruhi Performa:**
 - Akurasi skala warna (RGB, HSV).
 - Kualitas pencahayaan saat pengambilan gambar.
 - Proses **normalisasi** yang konsisten.

B. Bagaimana Perbedaan Akurasi Jika Jumlah Dataset Ditambah?

- **Dataset Lebih Besar:**
 - **Decision Tree:** Akurasi meningkat karena model mempelajari lebih banyak pola warna, tetapi risiko overfitting jika pohon terlalu dalam.
 - **KNN:** Lebih akurat karena tetangga terdekat lebih representatif, namun waktu prediksi bertambah karena lebih banyak data yang dihitung.
- **Kesimpulan:** Dataset yang lebih besar dan beragam (misalnya, warna di bawah pencahayaan berbeda) akan memberikan hasil klasifikasi yang **lebih stabil** dan **akurasi yang lebih tinggi**.

C. Bagaimana Cara Meningkatkan Kinerja Model Klasifikasi?

Beberapa metode untuk meningkatkan kinerja model:

- **Optimasi Parameter Model:**
 - **Decision Tree:** Sesuaikan kedalaman pohon (*max_depth*) dan minimal sampel per node (*min_samples_split*) untuk mencegah overfitting.
 - **KNN:** Pilih nilai *K* yang optimal menggunakan teknik validasi silang (**cross-validation**).
- **Feature Engineering:**
 - Gunakan **konversi warna** lain seperti **HSV** atau **LAB**, yang lebih robust terhadap variasi pencahayaan.
- **Peningkatan Dataset:**
 - Tambahkan **data augmentasi** seperti gambar dari berbagai kondisi pencahayaan dan latar belakang.
- **Normalisasi yang Konsisten:**
 - Pastikan semua data menggunakan **scaler** yang sama untuk menjaga konsistensi skala.
- **Pemilihan Algoritma Tambahan:**
 - Pertimbangkan algoritma seperti **Random Forest** atau **Support Vector Machine (SVM)** untuk klasifikasi warna yang lebih kompleks.

5. Assignment

- **Decision Tree menggunakan csv datasets**

Decision Tree adalah algoritma machine learning yang digunakan untuk klasifikasi dengan membuat struktur pohon keputusan berdasarkan fitur data. Dalam kasus deteksi warna, dataset dalam format CSV biasanya berisi nilai RGB dan label warna. Prosesnya dimulai dengan mengimpor dataset menggunakan pandas, kemudian melakukan normalisasi data dengan StandardScaler agar skala fitur seimbang. Selanjutnya, data dibagi menjadi set pelatihan dan

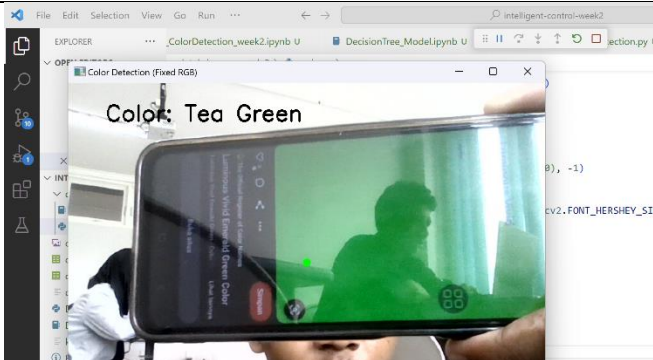
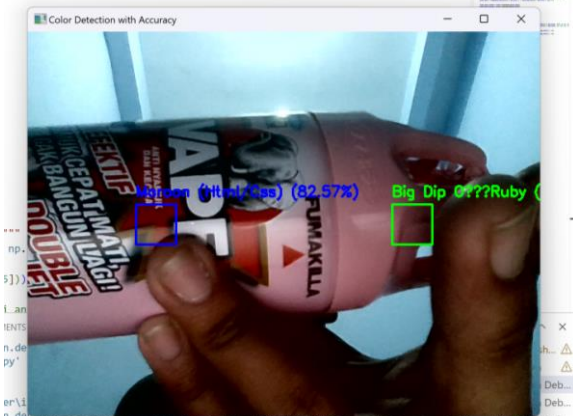
pengujian untuk melatih model DecisionTreeClassifier dari scikit-learn. Model kemudian dievaluasi menggunakan metrik seperti akurasi untuk mengukur kinerjanya. Dengan dataset yang beragam dan parameter model yang dioptimalkan, Decision Tree mampu mengklasifikasikan warna secara akurat dan efisien.

- **Decision Tree menggunakan training datasets**

Training dataset berupa gambar untuk Decision Tree dalam tugas klasifikasi warna berisi sekumpulan gambar yang mewakili berbagai warna dengan label yang sesuai. Setiap gambar dipecah menjadi data numerik, biasanya dalam bentuk nilai RGB (Red, Green, Blue) dari setiap pixel. Proses ini disebut ekstraksi fitur, di mana pixel atau area tertentu dari gambar dianalisis untuk mendapatkan nilai warna. Selain RGB, terkadang fitur tambahan seperti HSV (Hue, Saturation, Value) digunakan karena lebih sesuai untuk analisis warna dalam beberapa kasus.

Selama pelatihan, algoritma Decision Tree mempelajari pola dalam dataset dengan membangun struktur pohon berdasarkan pembagian data. Setiap node dalam pohon mewakili pertanyaan seperti "Apakah nilai Red > 100?" atau "Apakah nilai Blue < 50?", yang memandu pembagian dataset ke subgrup yang lebih homogen. Proses ini berlanjut hingga setiap daun pohon mewakili label warna tertentu, misalnya "Merah", "Hijau", atau "Biru".

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil Pengamatan
1	KNN menggunakan CSV (single color)	
2	Decision Tree menggunakan CSV (multicolor)	
3	Decision Tree menggunakan Training data set	

7. Kesimpulan

Pada praktikum "**Implementasi Machine Learning untuk Deteksi Warna Secara Real-Time Menggunakan Decision Tree dan KNN dengan OpenCV**", berhasil diterapkan dua algoritma pembelajaran mesin, yaitu **Decision Tree** dan **K-Nearest Neighbors (KNN)**, untuk mendeteksi warna secara langsung melalui kamera. Proses deteksi dimulai dengan **pengumpulan dan pelabelan data warna** dari gambar, di mana nilai RGB dari pixel digunakan sebagai fitur utama. Data ini kemudian digunakan untuk melatih kedua model.

Hasil praktikum menunjukkan bahwa **Decision Tree** memberikan hasil prediksi yang cepat dan mudah dipahami karena alur keputusannya yang jelas berbasis aturan if-else. Sementara itu, **KNN** memberikan performa yang cukup akurat dengan membandingkan data pixel baru terhadap tetangga terdekatnya, meskipun memerlukan waktu pemrosesan yang lebih lama saat prediksi karena bergantung pada keseluruhan dataset.

Dengan bantuan **OpenCV**, deteksi warna ditampilkan secara **real-time** menggunakan **bounding box** dan label warna beserta tingkat akurasi. Praktikum ini membuktikan bahwa integrasi machine learning dengan computer vision dapat memberikan solusi efektif untuk deteksi warna secara langsung, di mana **Decision Tree** unggul dalam kecepatan dan interpretasi, sementara **KNN** menawarkan ketahanan terhadap noise dengan akurasi yang baik.

.

8. Saran

Saran untuk praktikum ini adalah meningkatkan kualitas dataset dengan menambahkan lebih banyak variasi warna dan kondisi pencahayaan agar model lebih robust. Selain itu, dapat dicoba algoritma lain seperti Random Forest atau SVM untuk membandingkan performa dengan Decision Tree dan KNN. Terakhir, optimalkan preprocessing seperti normalisasi dan pengurangan noise untuk meningkatkan akurasi deteksi warna secara real-time..

9. Daftar Pustaka

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
OpenCV Documentation. (2023). "Object Detection using Haar Cascades."
<https://docs.opencv.org>.

Raschka, S., & Mirjalili, V. (2022). *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2* (3rd ed.). Packt Publishing.

Zhang, X., & Wang, S. (2021). Real-Time Color Detection and Classification Using Machine Learning Techniques. *International Journal of Computer Applications*, 183(10), 15-22.